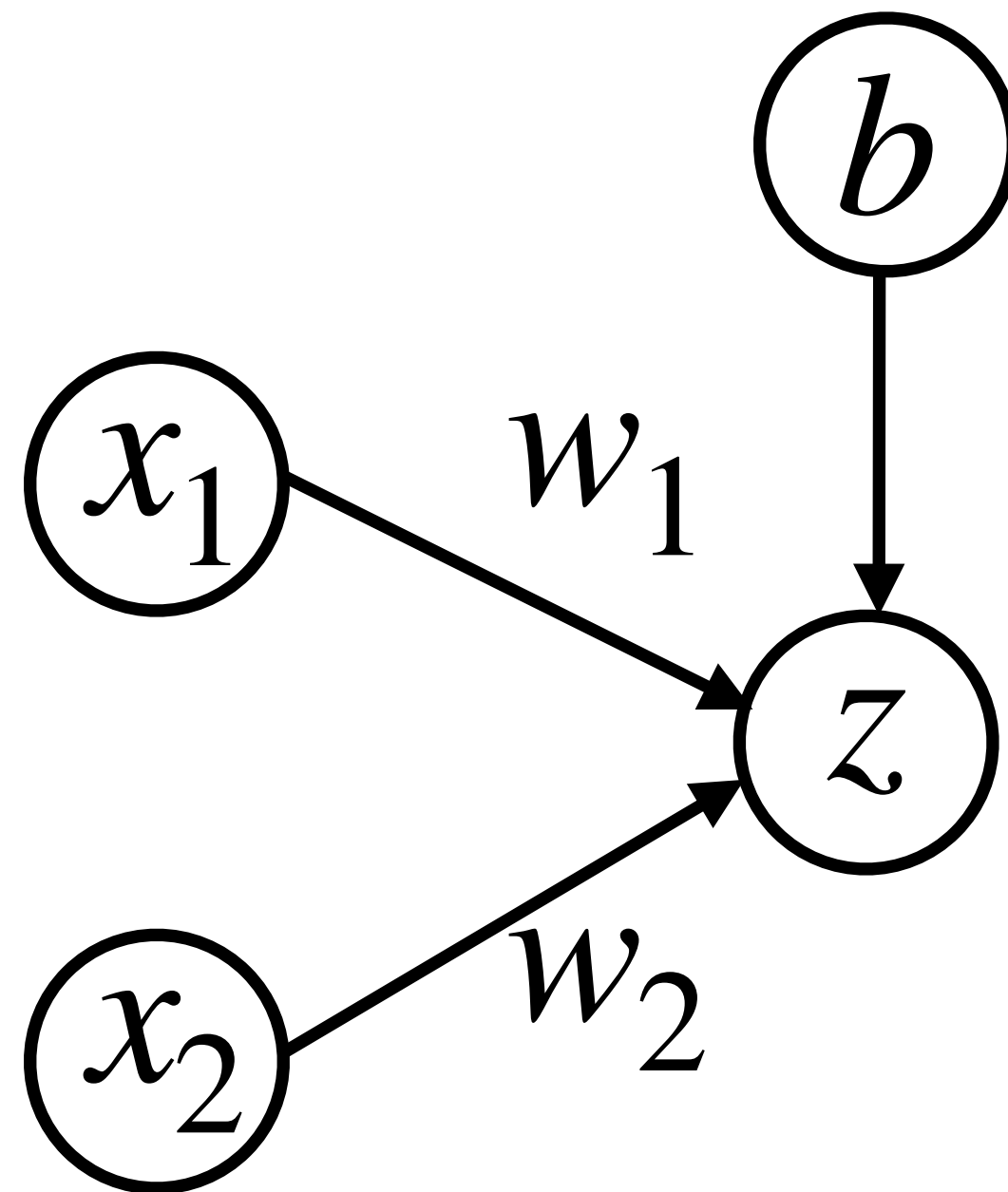


Feedforward Neural Networks

CSC 487: Deep Learning
Instructor: Jonathan Ventura

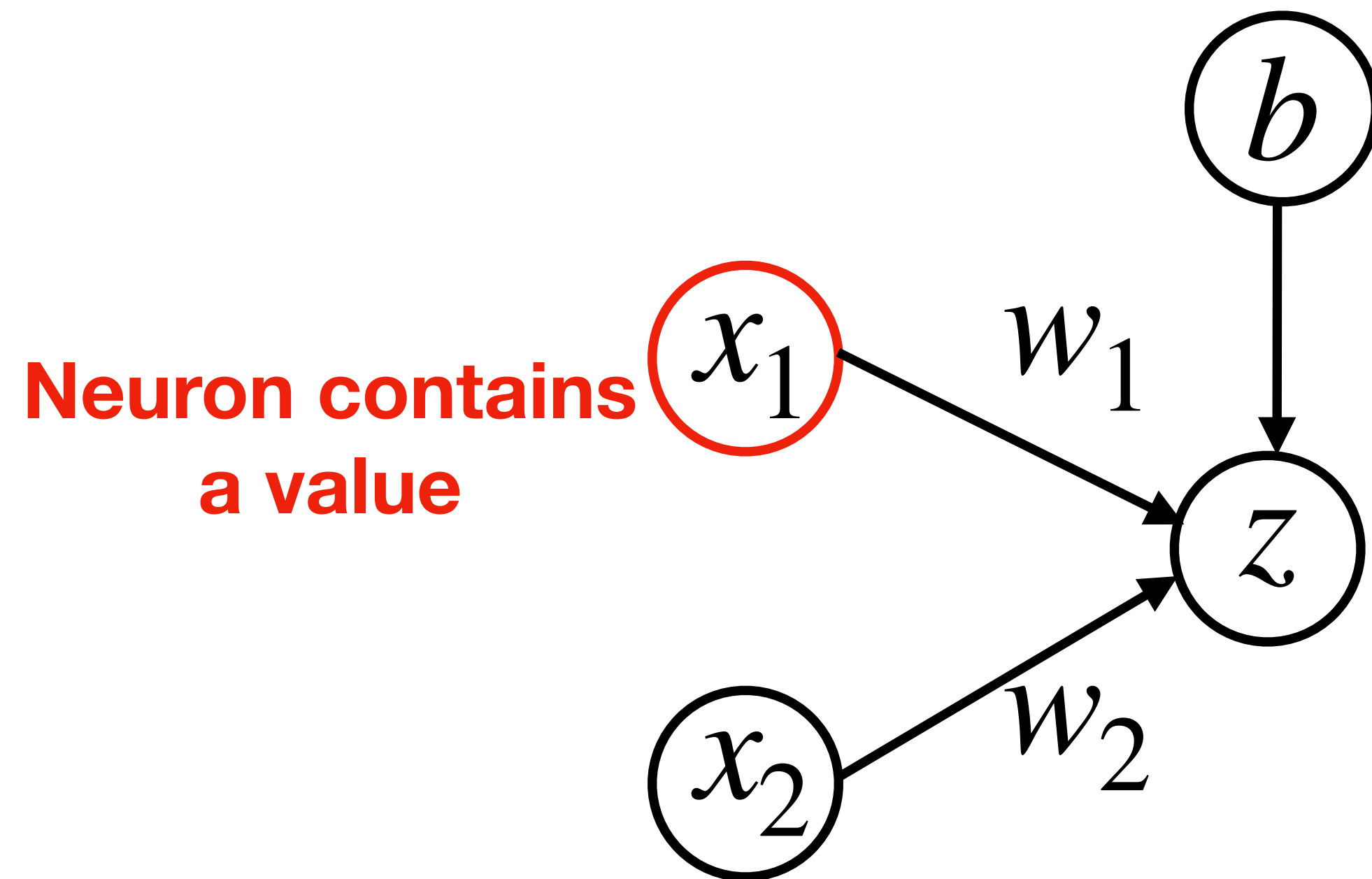
Perceptron

- The perceptron is a simple version of a **neural network**
- It is the same as a linear classifier as we have discussed.



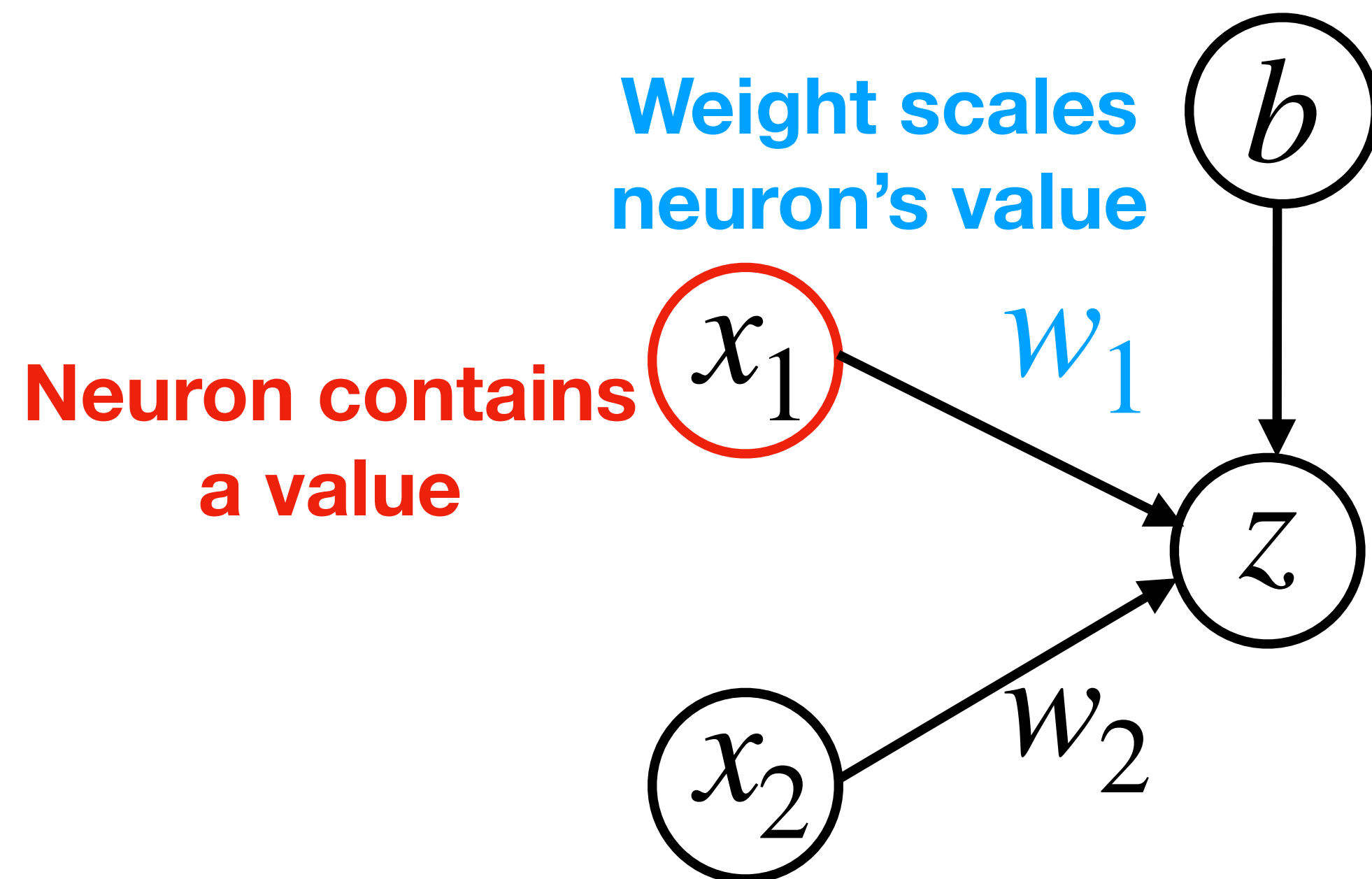
Perceptron

- The perceptron is a simple version of a **neural network**
- It is the same as a linear classifier as we have discussed.



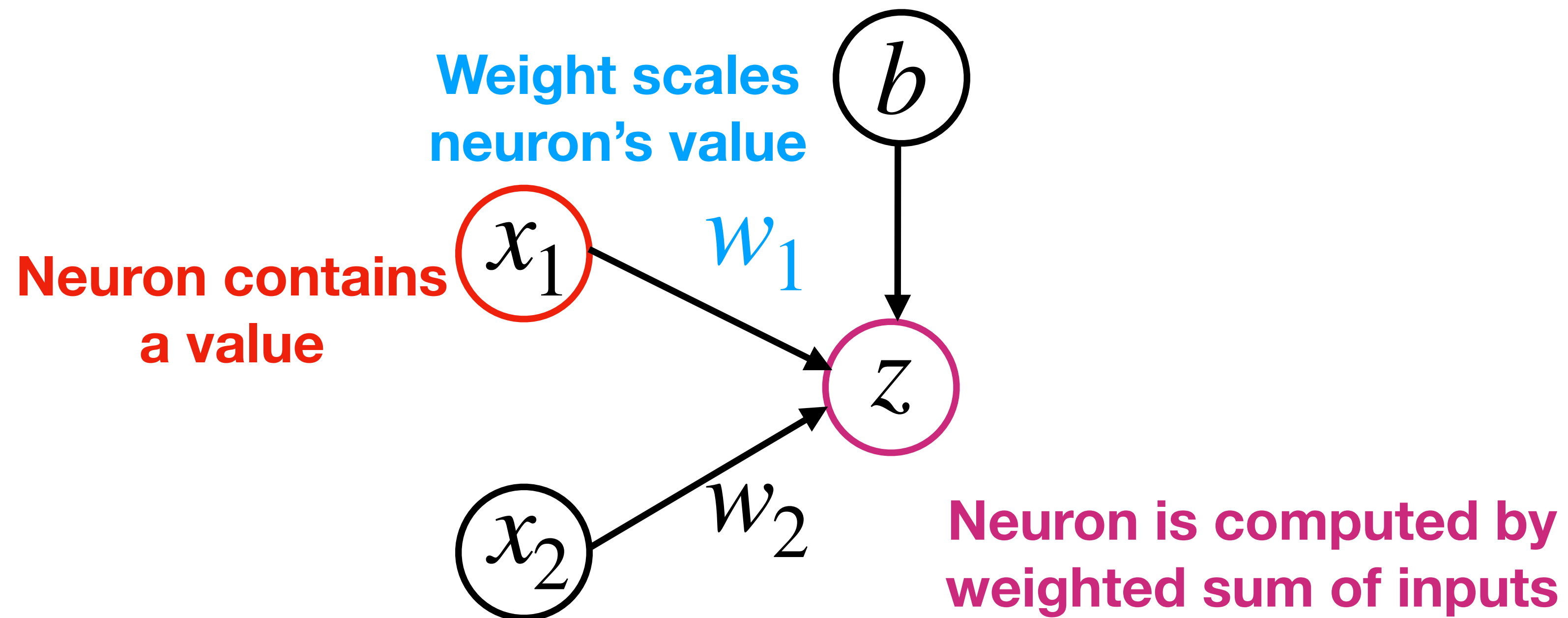
Perceptron

- The perceptron is a simple version of a **neural network**
- It is the same as a linear classifier as we have discussed.



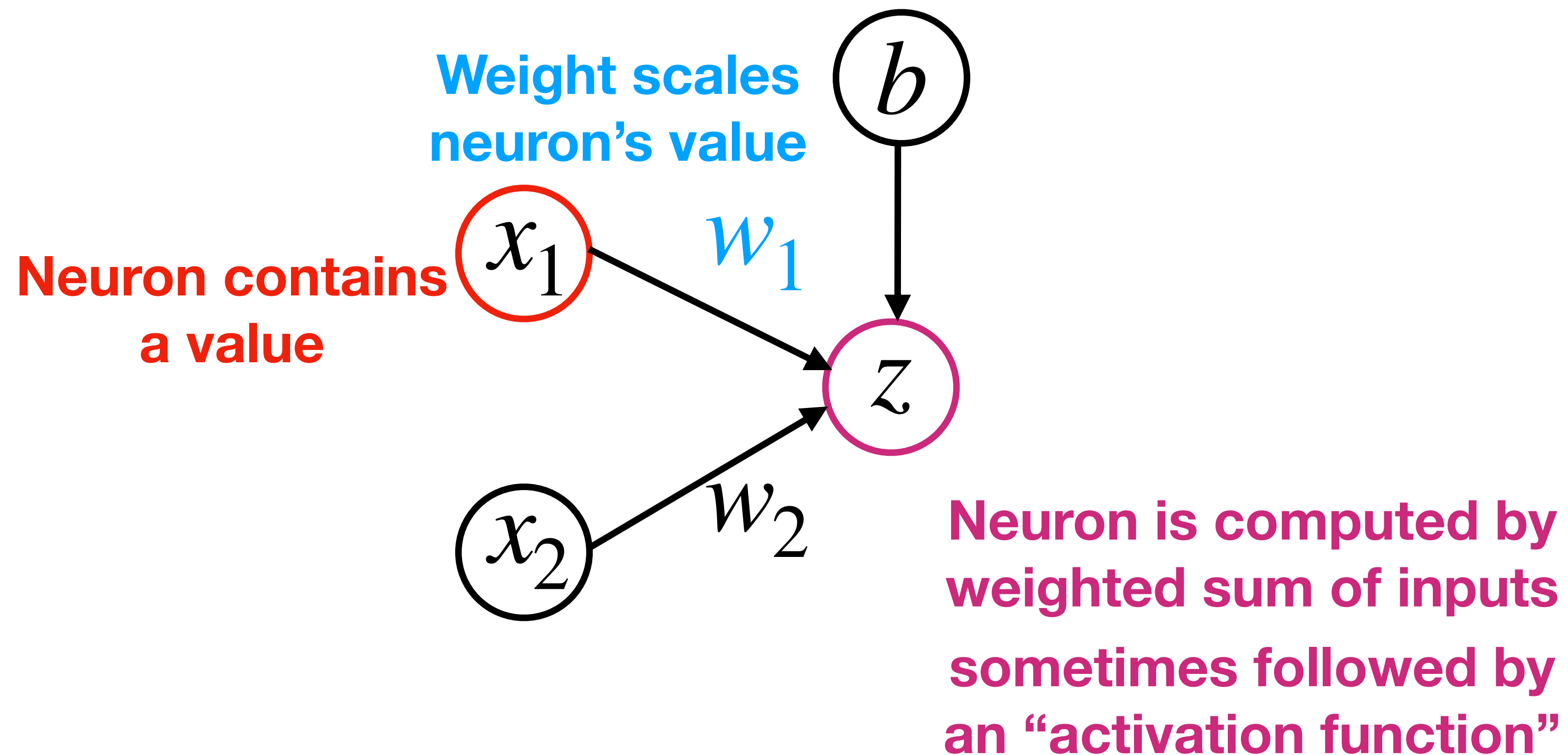
Perceptron

- The perceptron is a simple version of a **neural network**
- It is the same as a linear classifier as we have discussed.



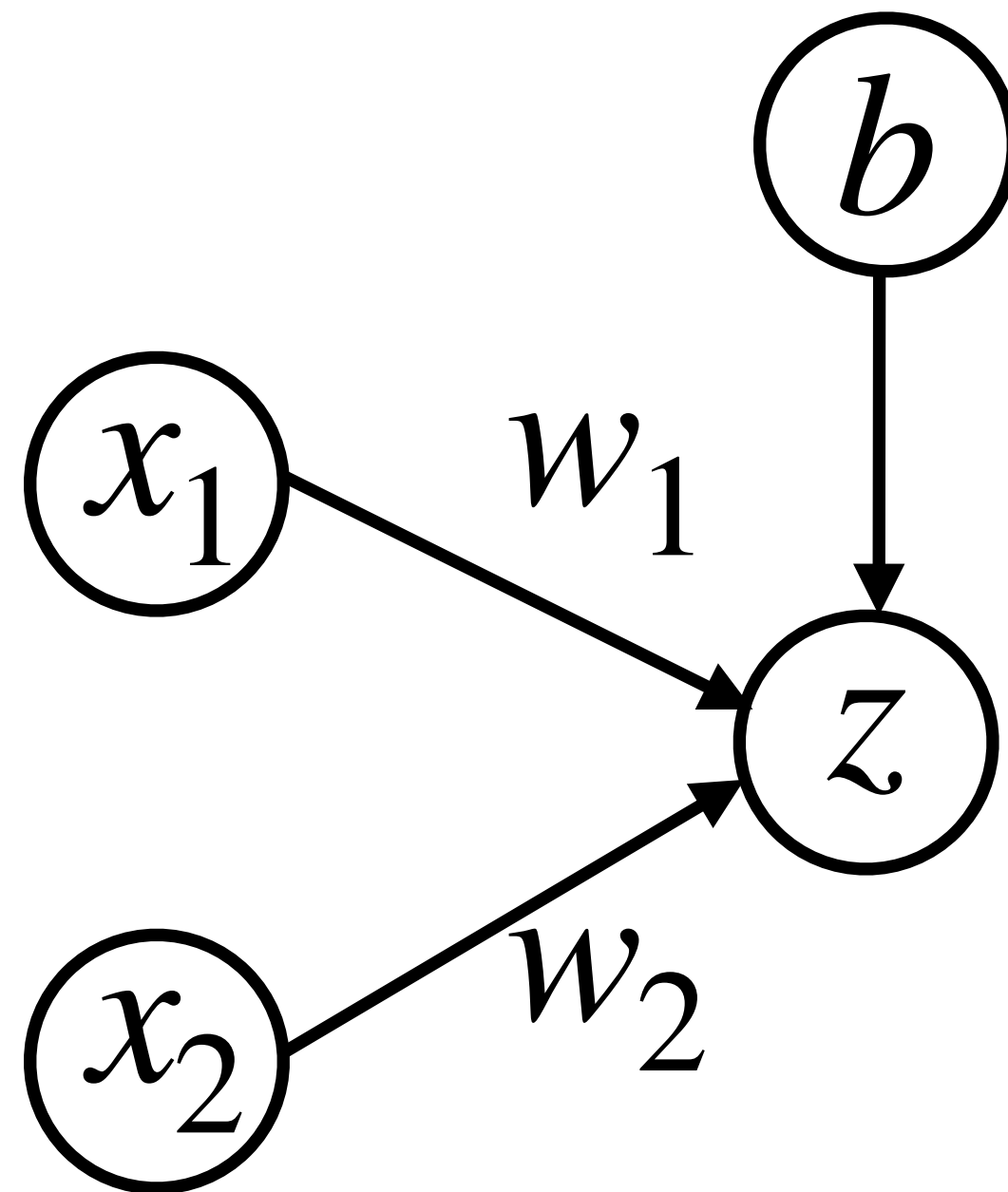
Perceptron

- The perceptron is a simple version of a **neural network**
- It is the same as a linear classifier as we have discussed.



Perceptron

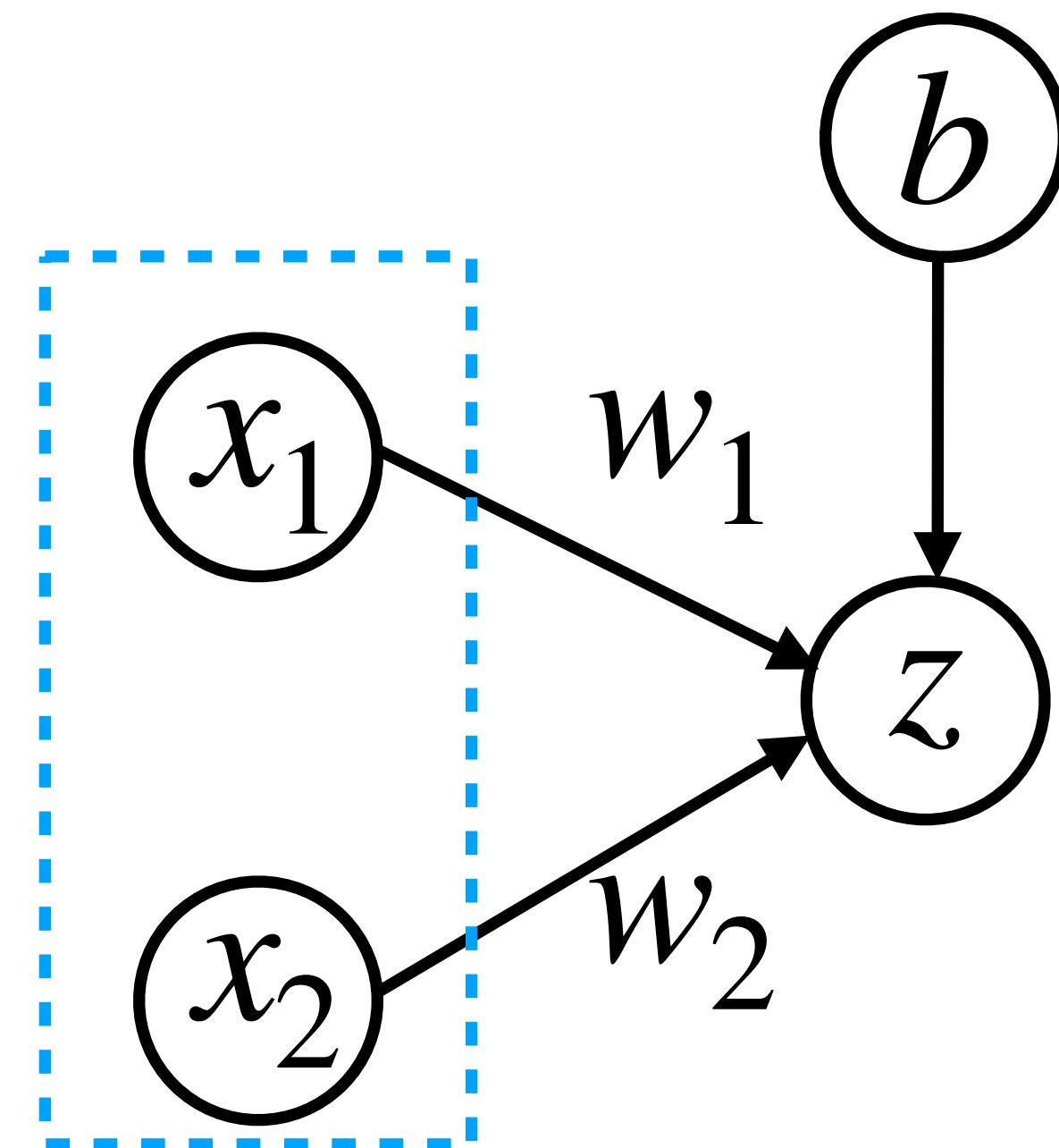
- The perceptron is a simple version of a **neural network**
- It is the same as a linear classifier as we have discussed.



$$z = w_1x_1 + w_2x_2 + b$$

Single-Layer Perceptron

- Single-layer perceptron is the simplest neural network

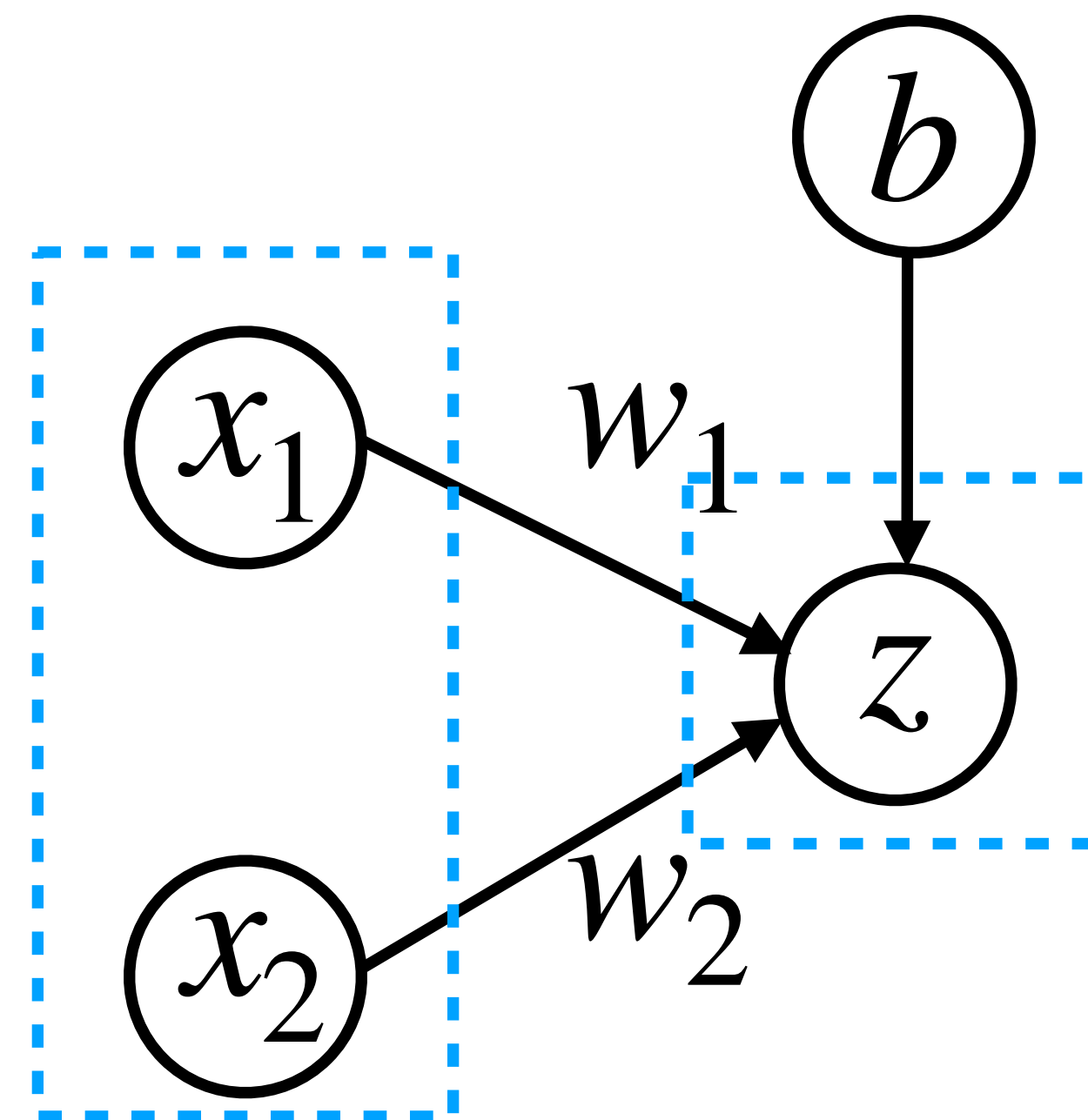


Input layer

X

Single-Layer Perceptron

- Single-layer perceptron is the simplest neural network



Input layer

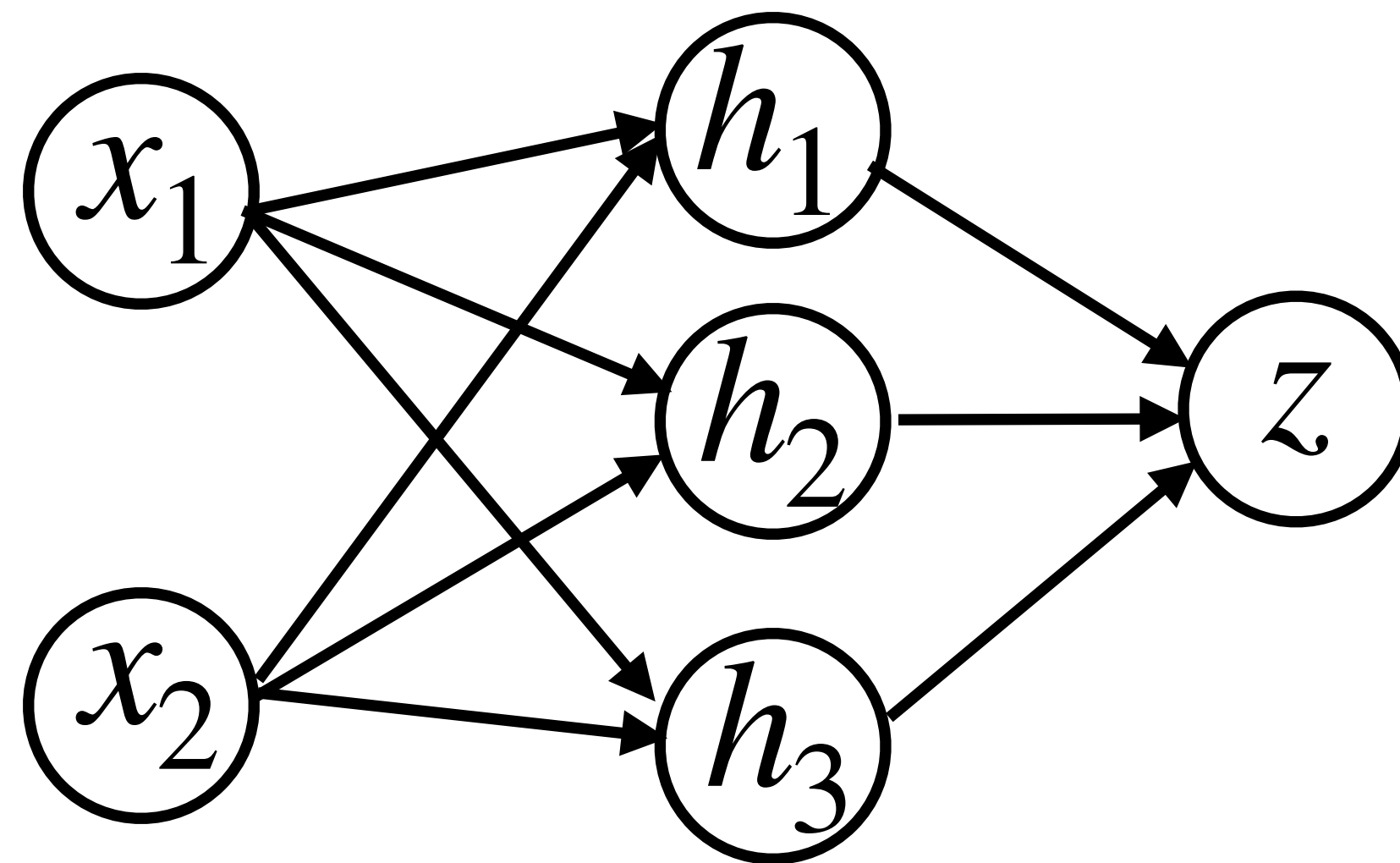
Output layer

$\mathbf{x}$

$$z = \mathbf{w}^T \mathbf{x} + b$$

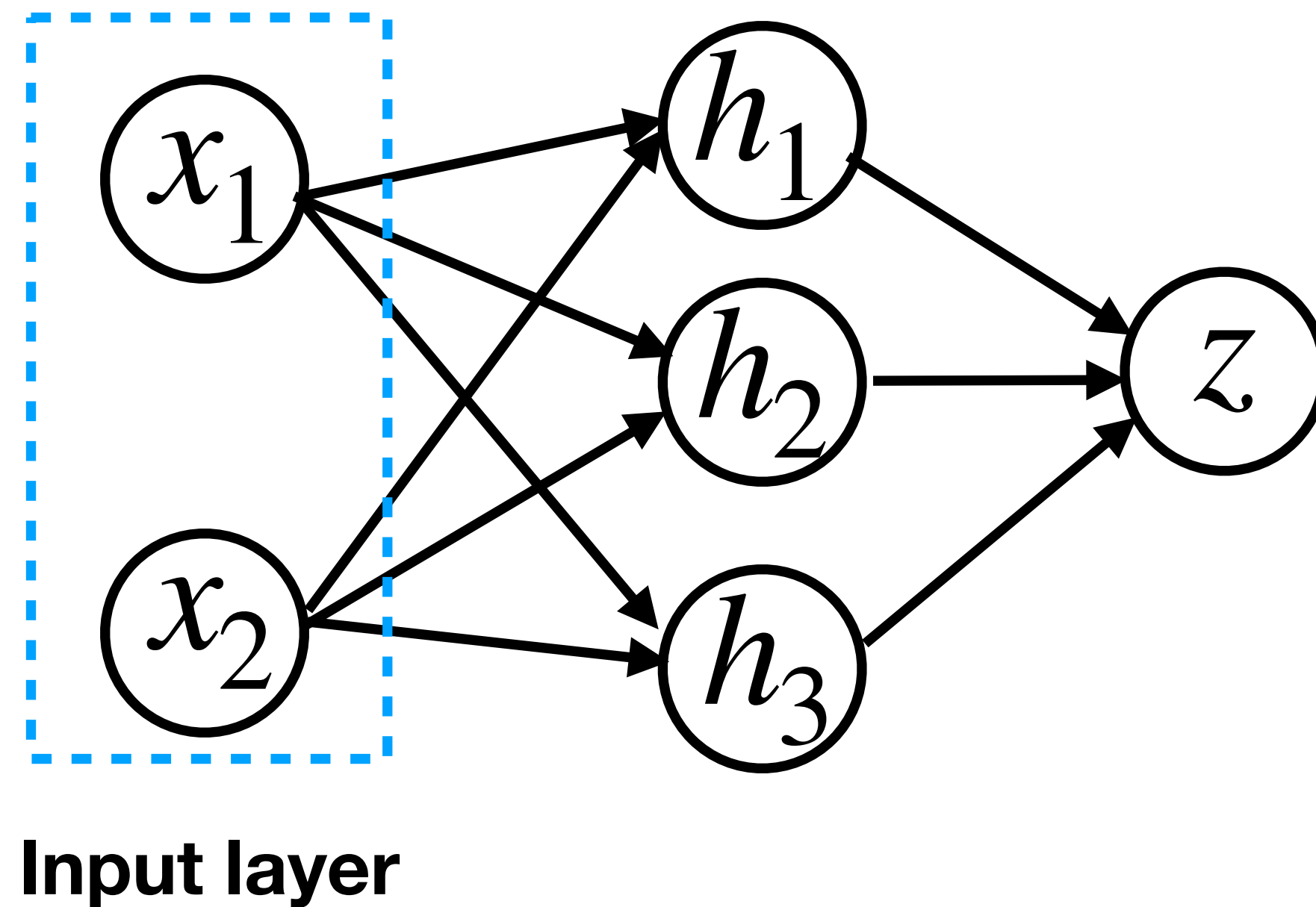
Multi-Layer Perceptron

- Perceptrons become more capable when we add “hidden” layers:



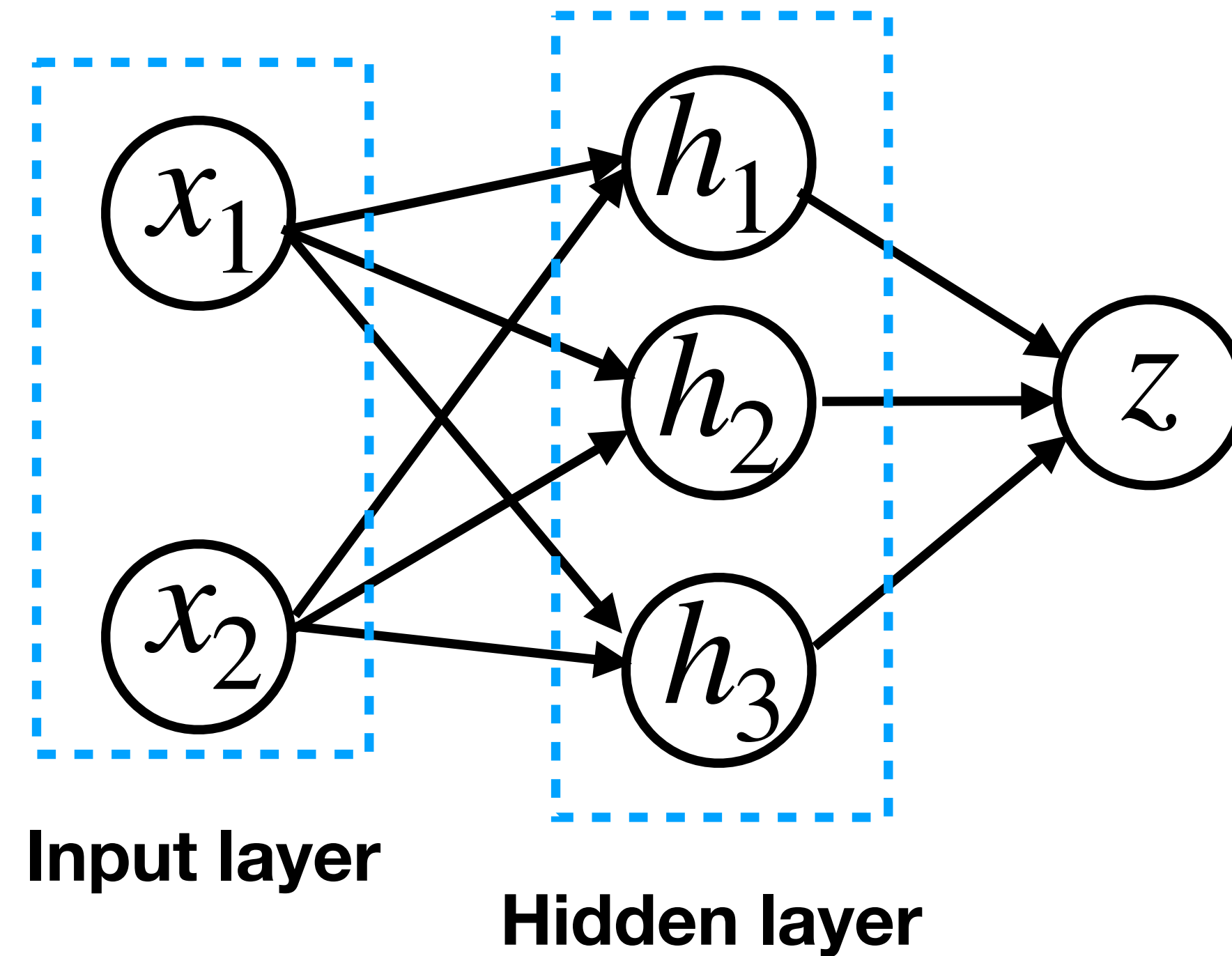
Multi-Layer Perceptron

- Perceptrons become more capable when we add “hidden” layers:



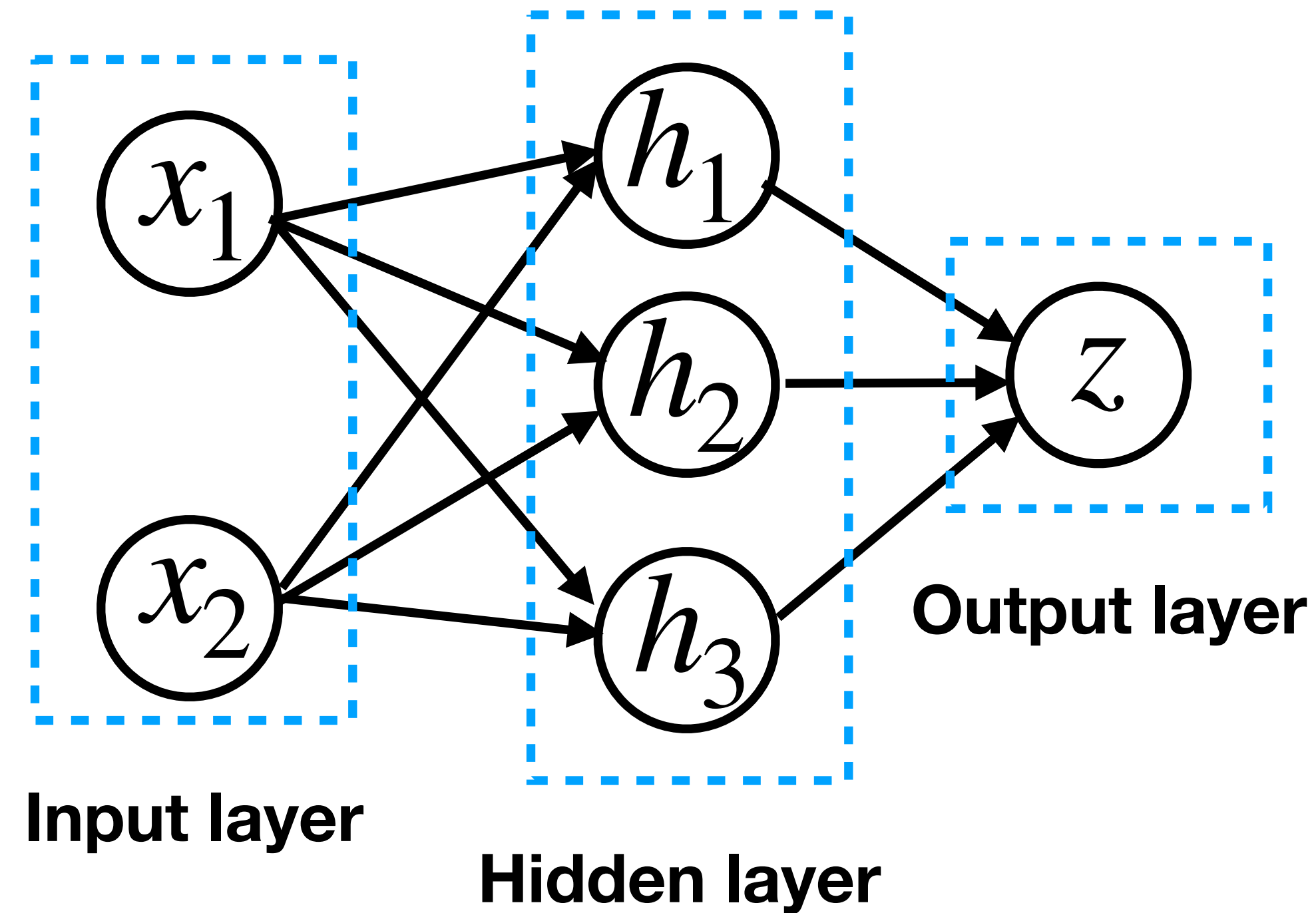
Multi-Layer Perceptron

- Perceptrons become more capable when we add “hidden” layers:



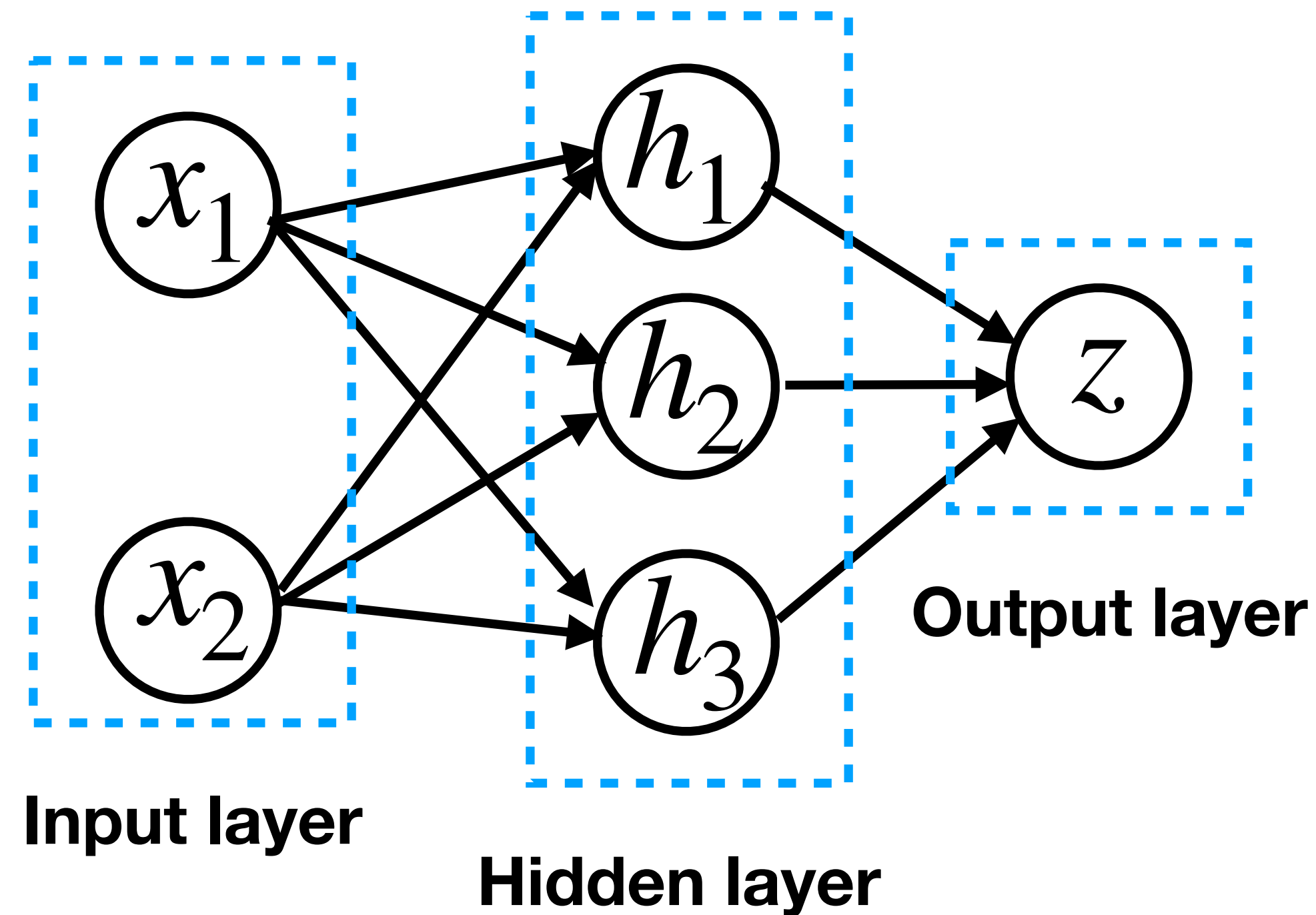
Multi-Layer Perceptron

- Perceptrons become more capable when we add “hidden” layers:



Multi-Layer Perceptron

- Perceptrons become more capable when we add “hidden” layers:



Activation function
(such as ReLU)

$$h = \sigma(\mathbf{w}_1^T \mathbf{x} + \mathbf{b}_1)$$

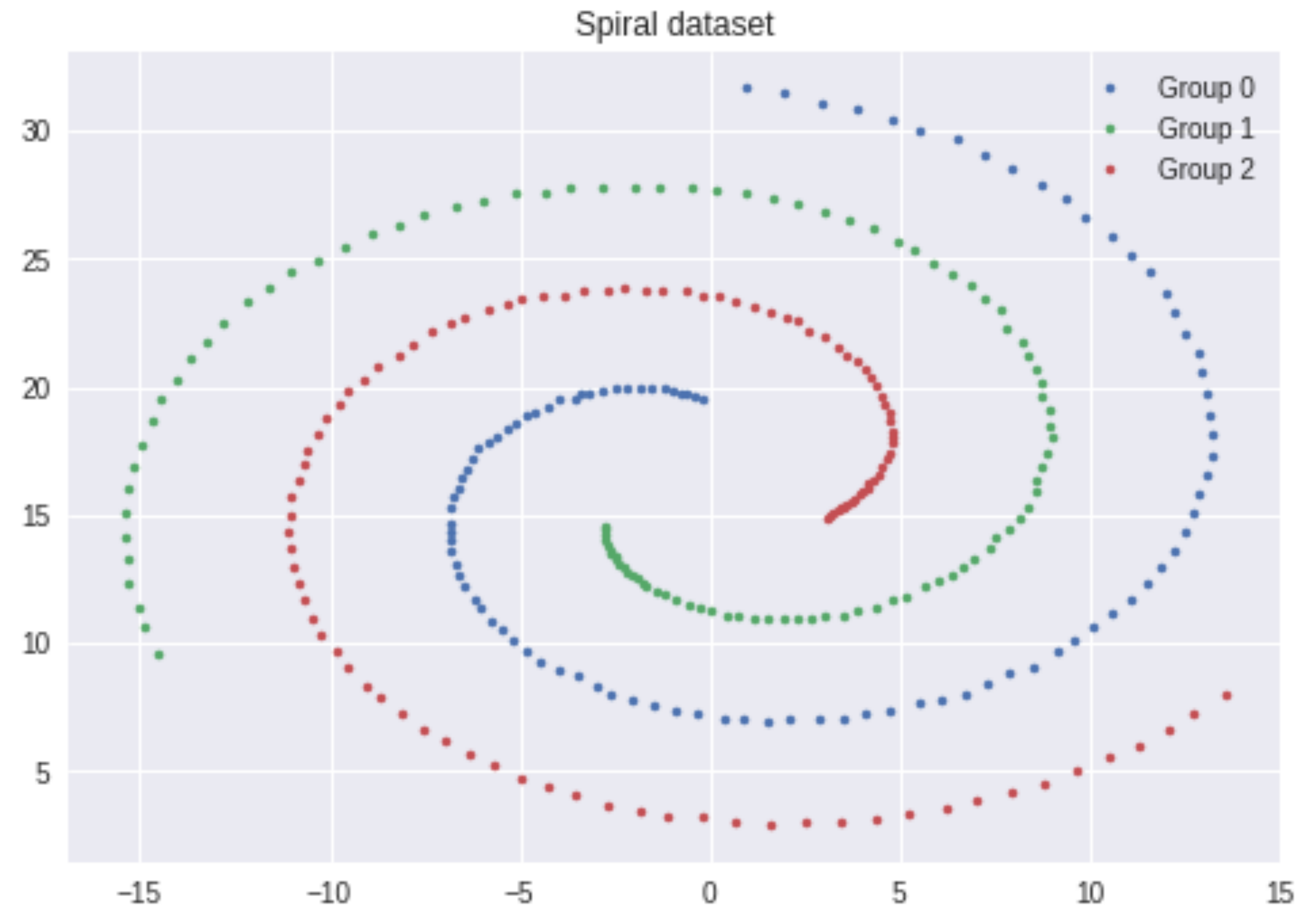
$$z = \mathbf{w}_2^T h + b_2$$

Example: Linear Classifier

- Goal: learn to separate 2D points into three spiral clusters

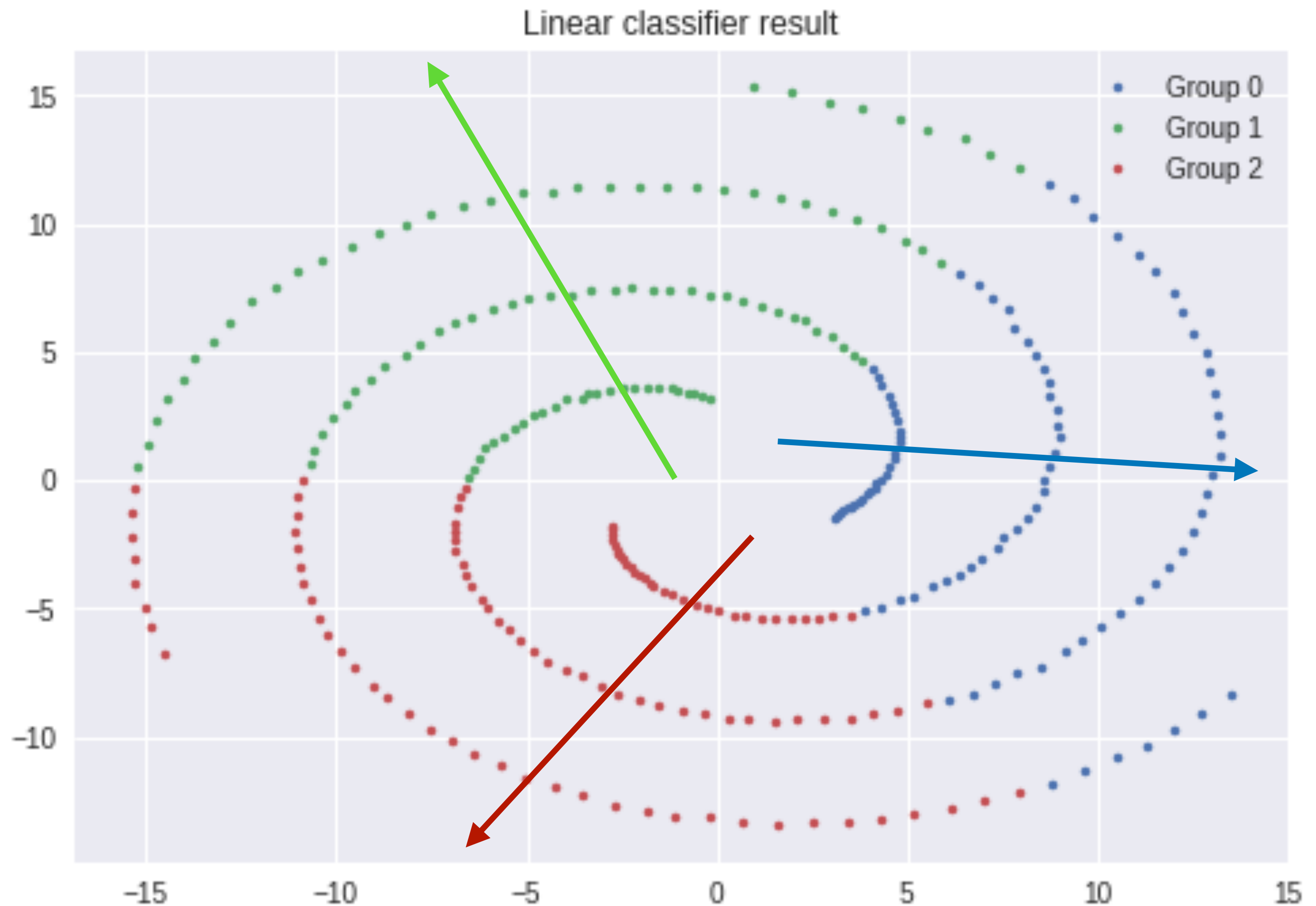
$$y = f^*(\mathbf{x})$$

- $\mathbf{x}$ is 2D point coordinates
(2x1 vector)
- y is group number (0,1, or 2)



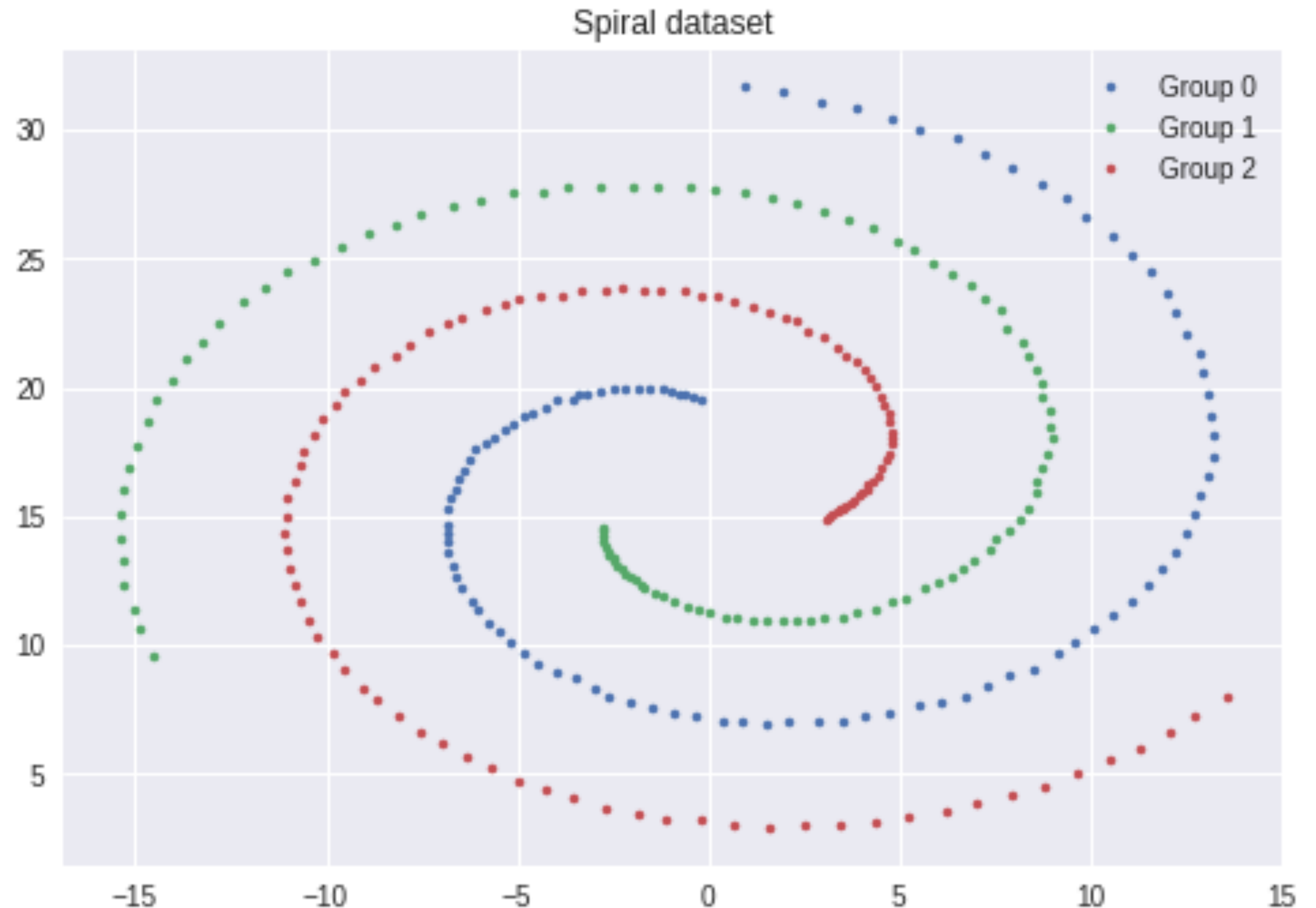
Linear classifier can't separate spirals

- This configuration of points is not “linearly separable”
- Accuracy is about 30%
- What do the linear classifier basis vectors look like?



Learning more complex functions

- How can we make a more powerful model?
- We need a **non-linear** transformation since linear transformation is not sufficient

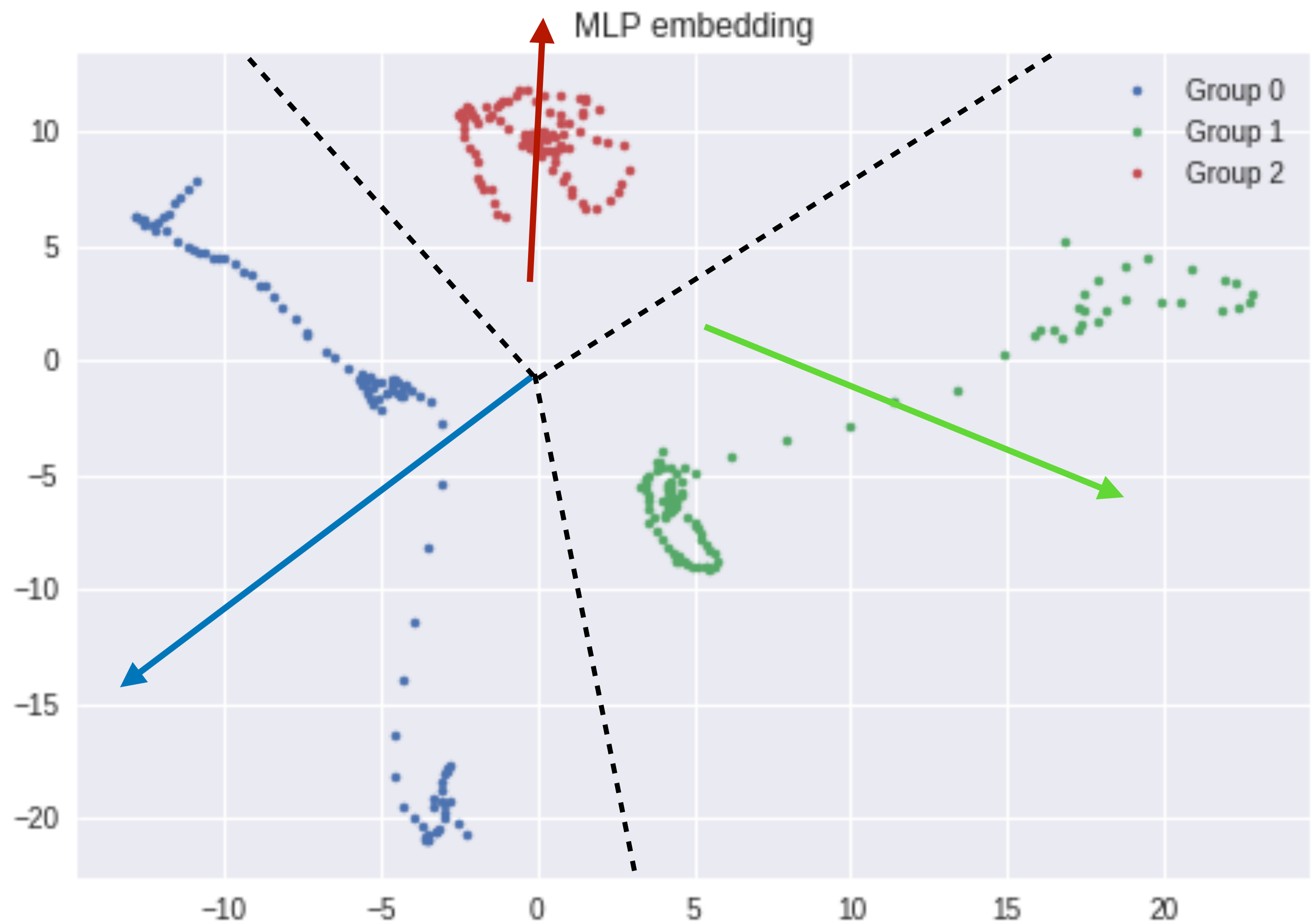


We can learn a better representation for linear classification

- Idea: learn an initial non-linear transformation before the linear classifier is applied

$$\mathbf{z} = \mathbf{W}\phi(\mathbf{x}) + \mathbf{b}$$

- $\Phi(\mathbf{x})$ maps the raw data $\mathbf{x}$ to a representation which is linearly separable



Feedforward Neural Network Composes Linear Transformations with Non-Linear Activations

- There are many options for the non-linear transformation $\Phi(\mathbf{x})$

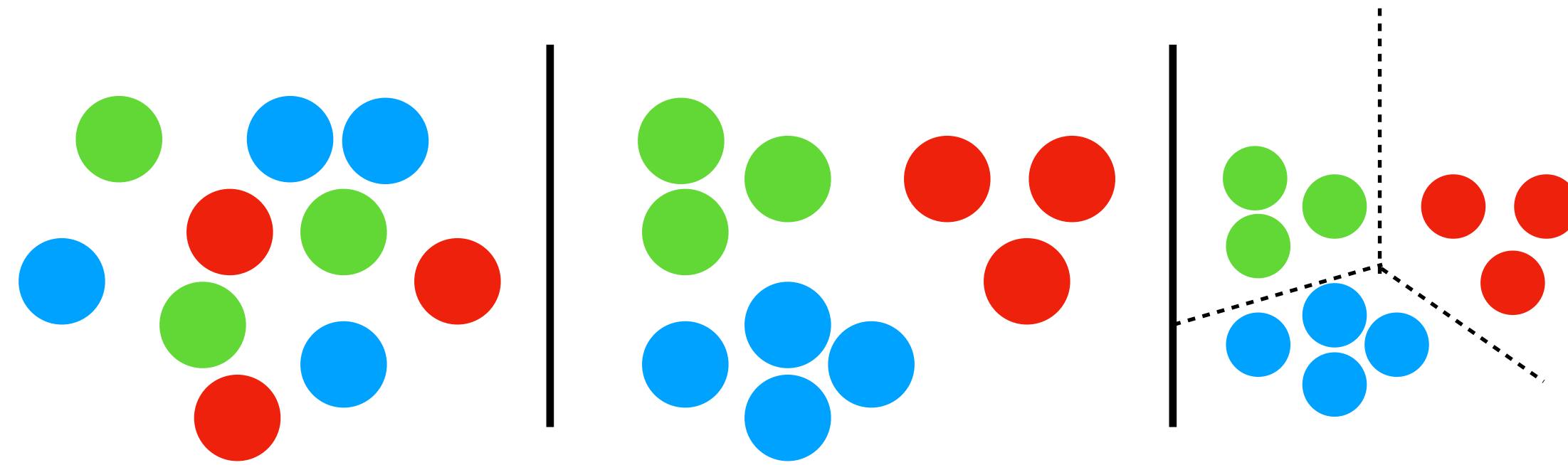
$$\mathbf{z} = \mathbf{W}\phi(\mathbf{x}) + \mathbf{b}$$

- Basic concept of a feedforward neural network is to combine an affine transformation with a non-linear “activation” function $\sigma(\cdot)$

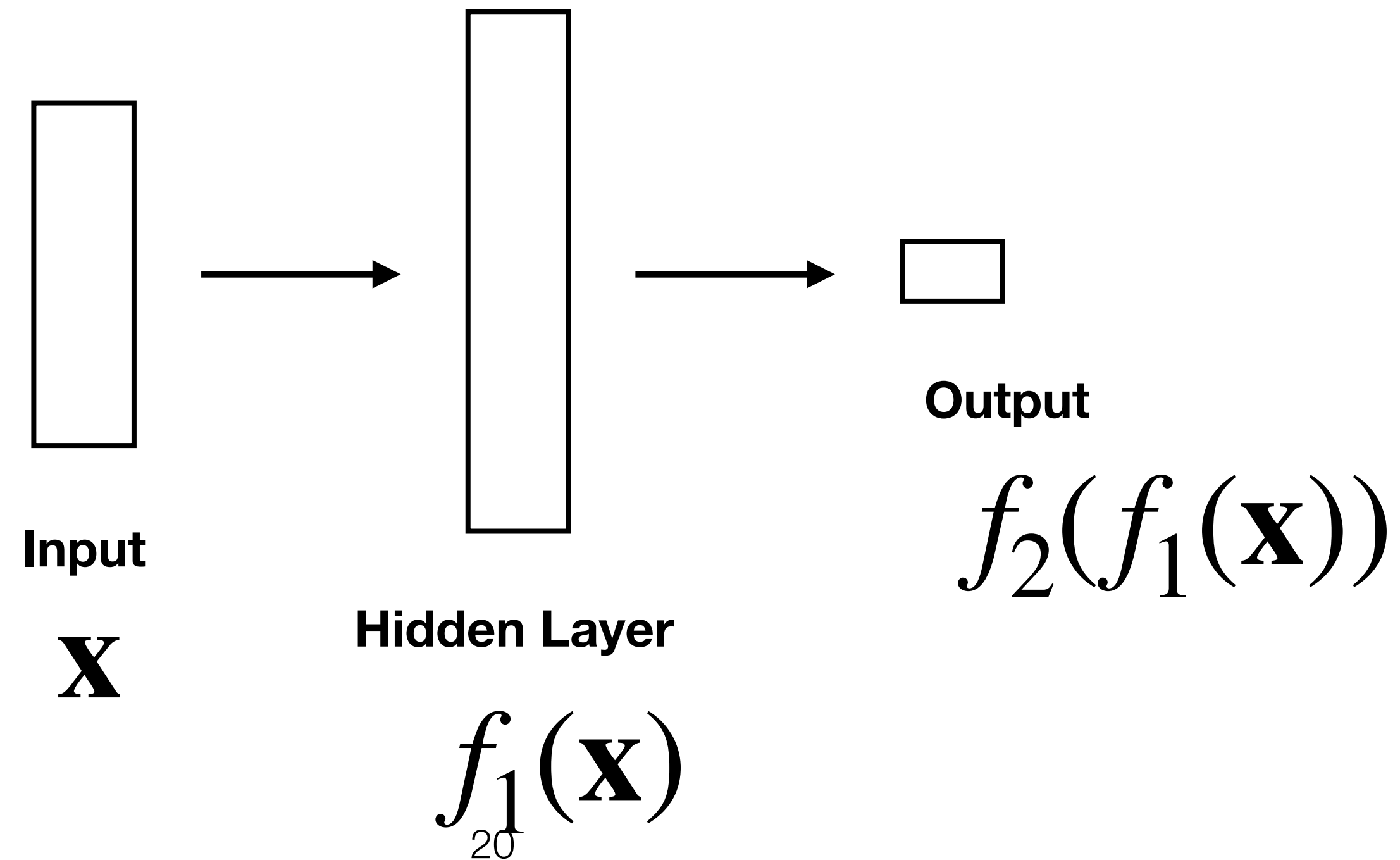
$$\phi(\mathbf{x}) = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{z} = \mathbf{W}_2\sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

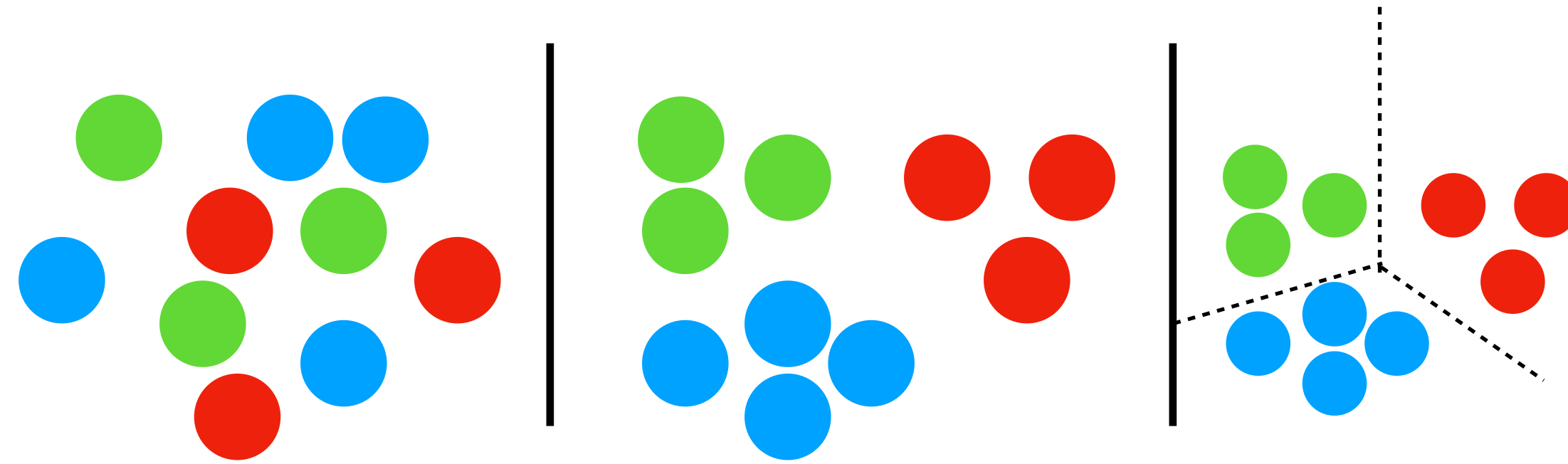
Feedforward NN is Chain of Transformations



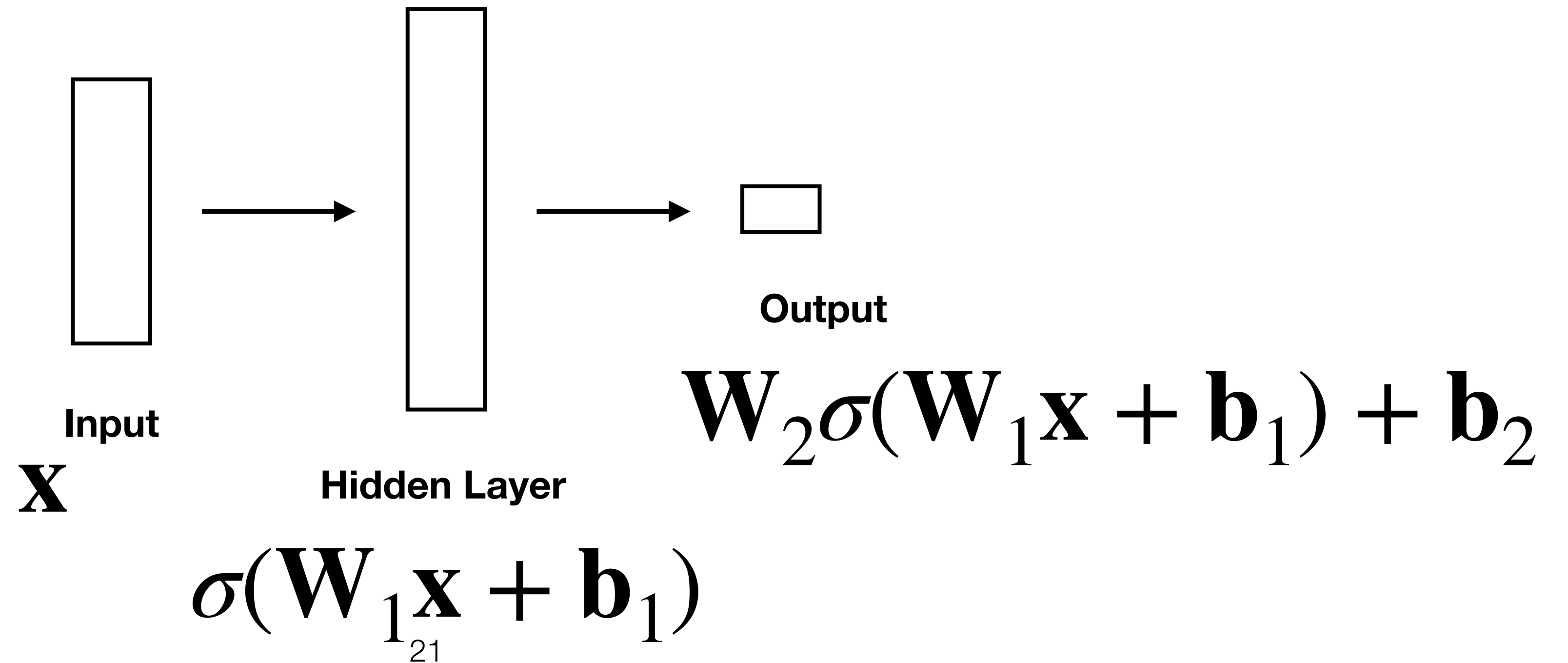
Feedforward Neural Network



Feedforward NN is Chain of Transformations

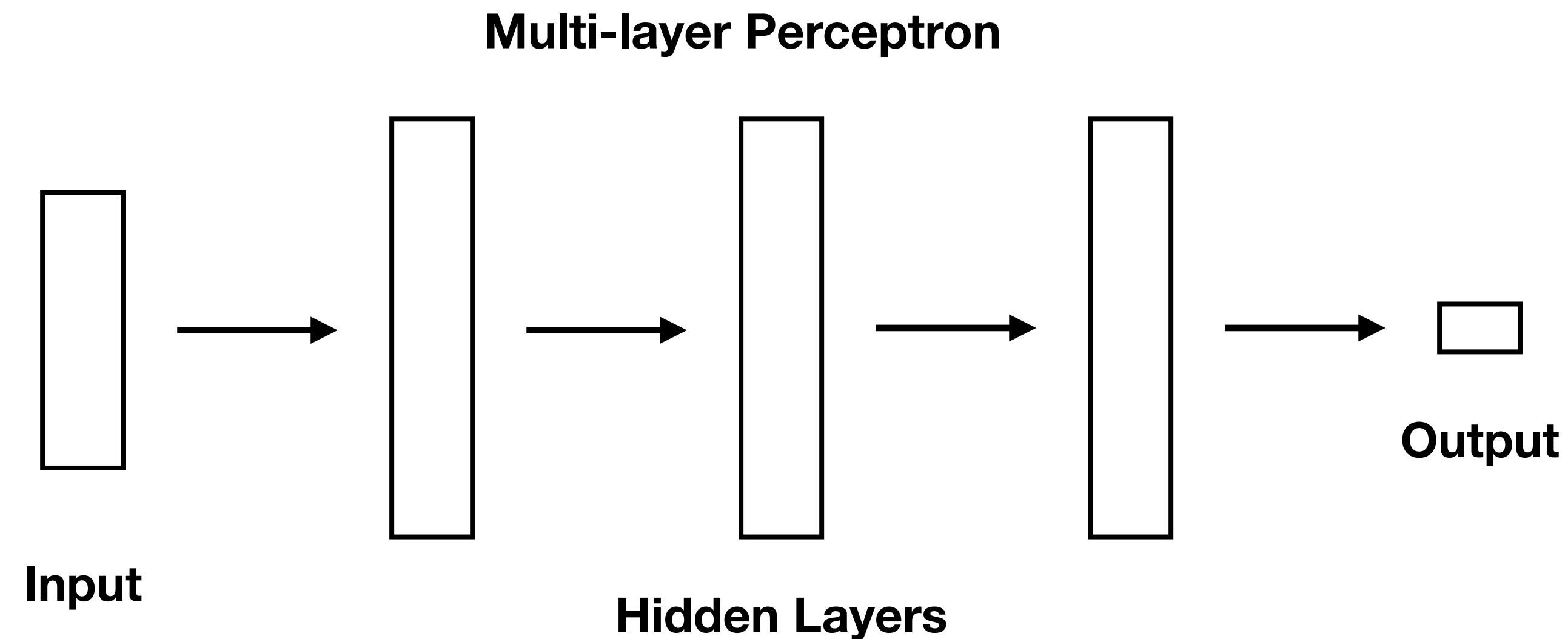


Feedforward Neural Network



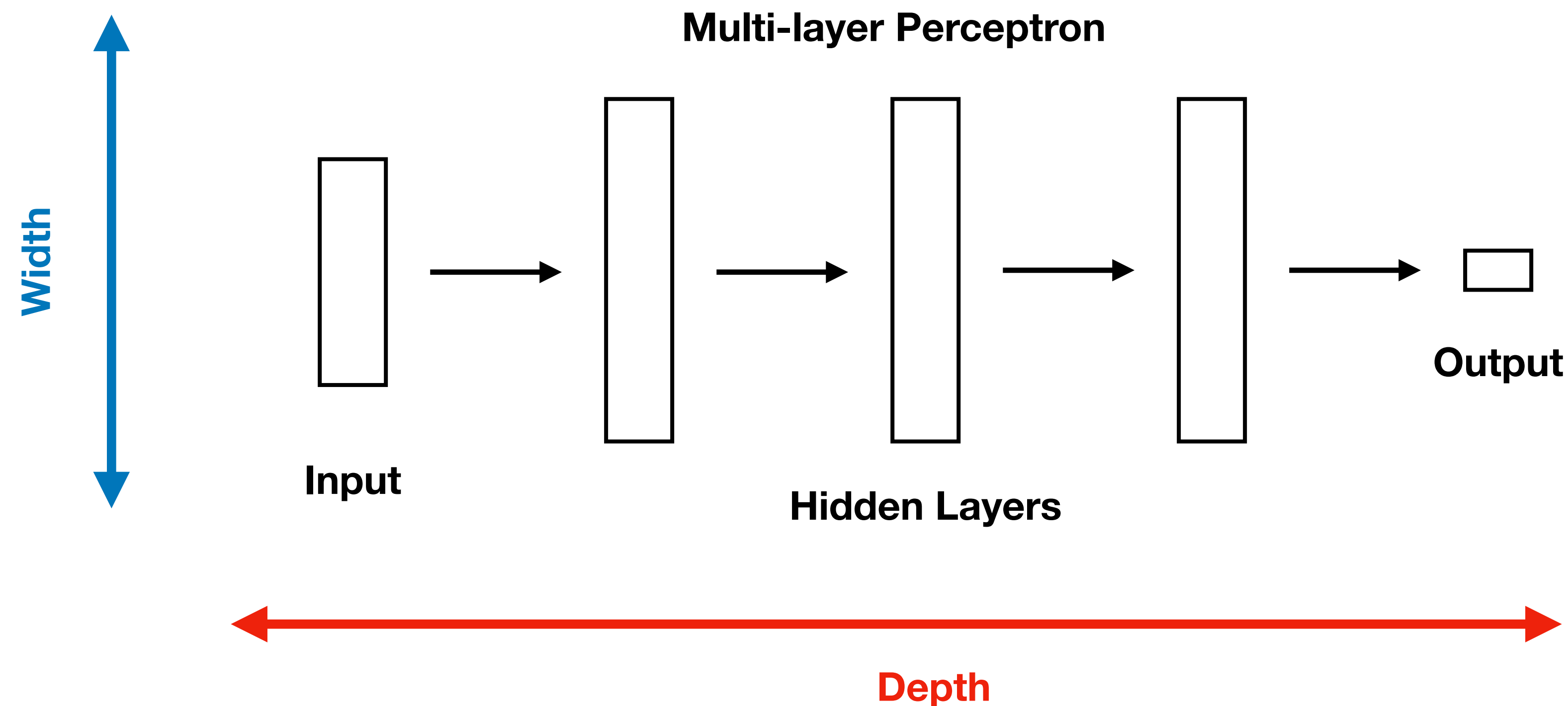
MLP has multiple hidden layers

- We can add as many hidden layers as we like
- By composing more transformations, we get a more complex / powerful model



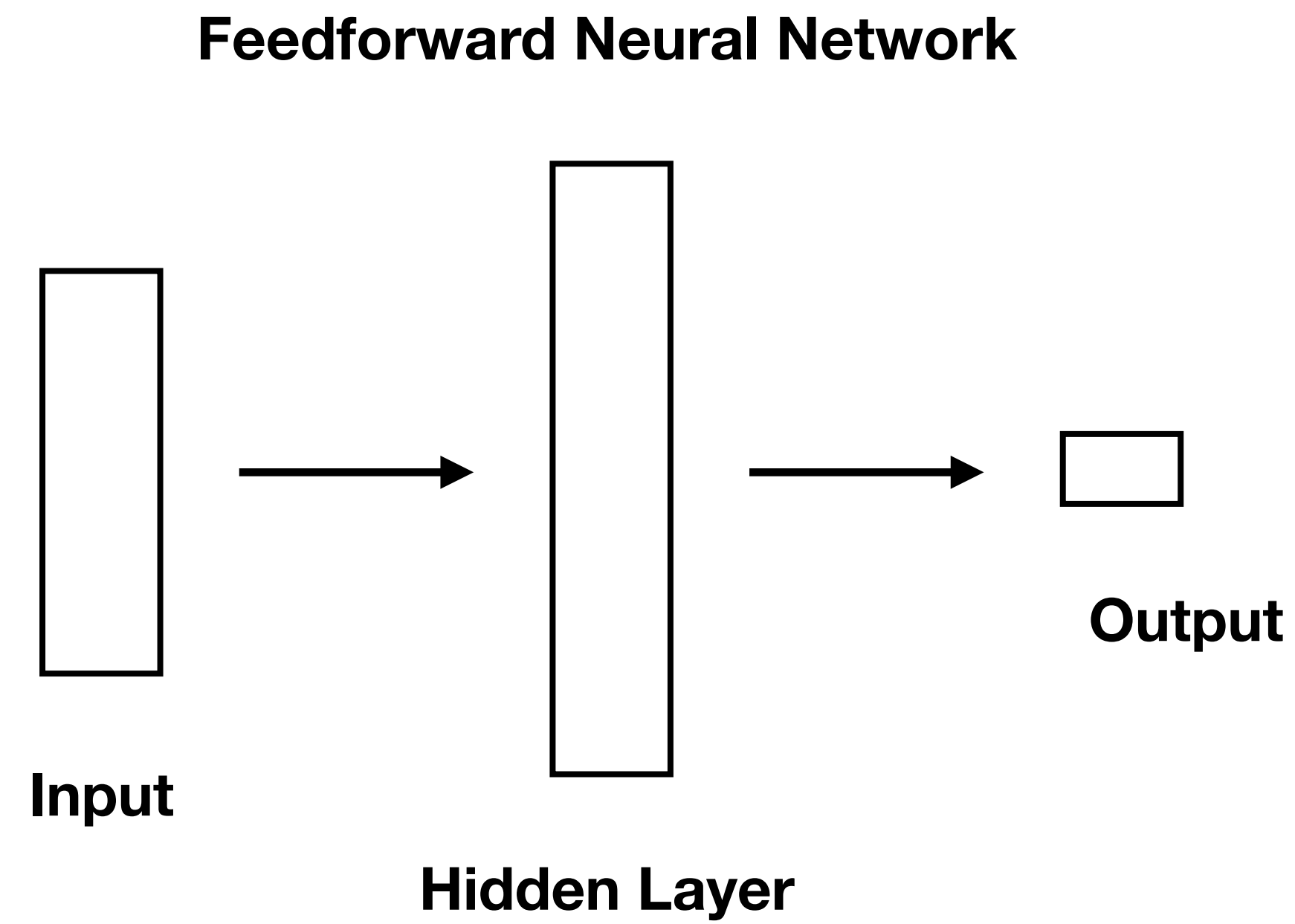
Deep learning = many hidden layers

- Number of layers is “depth” of network
- Size of vector in hidden layer is “width” of layer



Activation functions

Activation function provides non-linearity

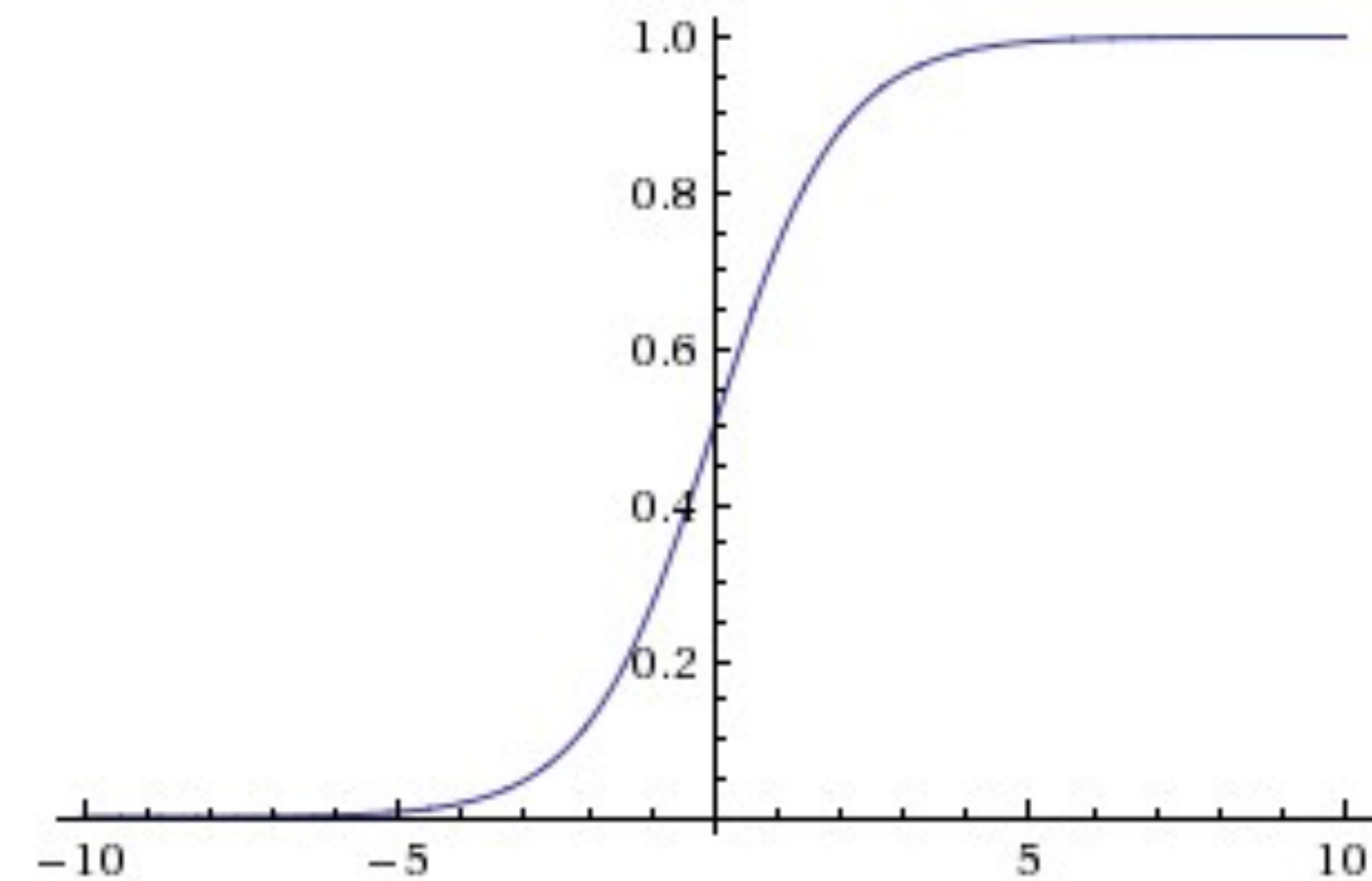


$$\mathbf{W}_2 g(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

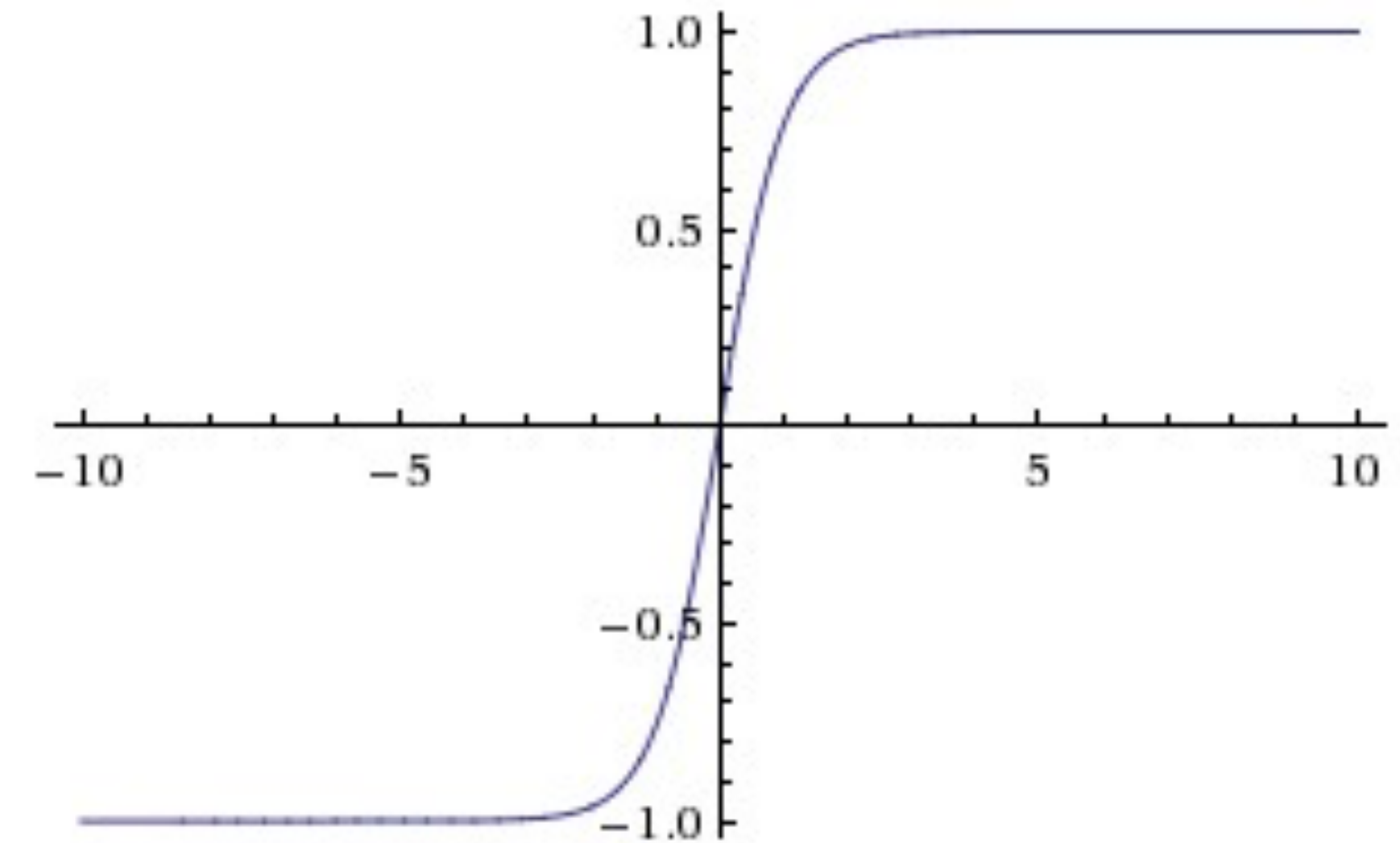
- Classical choice
- Not used in modern networks
 - Not zero-centered
 - Saturates (flattens out) at extremes



Tanh: hyperbolic tangent

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

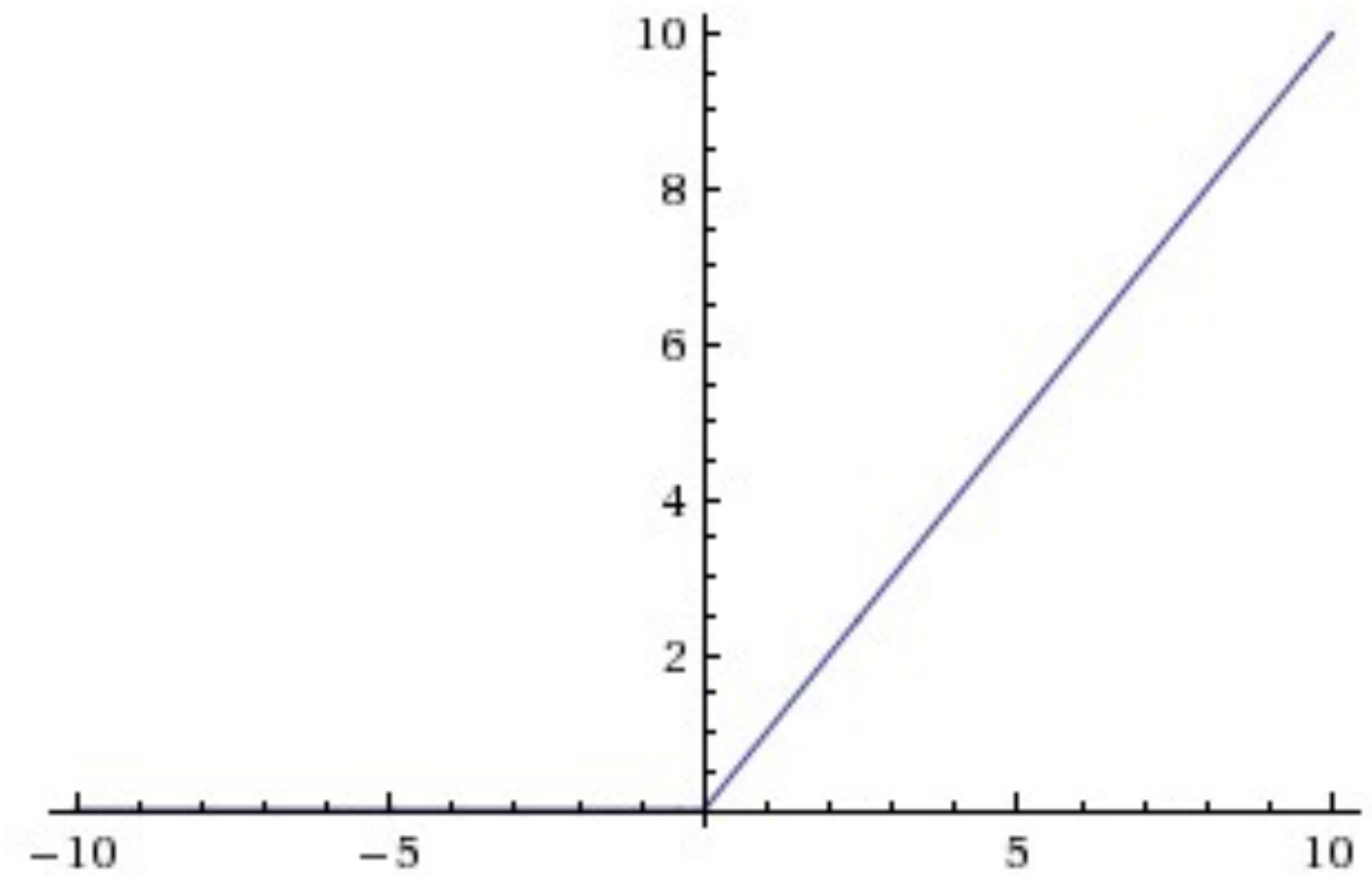
- Like a scaled and shifted sigmoid
- Better than sigmoid: zero-centered



ReLU: Rectified Linear Unit

$$\text{relu}(x) = \max(0, x)$$

- Part of deep learning revolution ca. 2011
- Empirically shown to speed up training, improve accuracy



GeLU / SiLU / Swish

$$\text{swish}(x) = x\sigma(\beta x)$$

- Empirically shown to outperform ReLU
- Used in many modern networks

